

SQL for Data Analytics - Assessment File

TOPS TECHNOLOGIES DATA ANALYTICS [A1]

Section A: Concept Application

1. SELECT * vs. Specifying Column Names

- **Functional Difference:** `SELECT *` retrieves all available columns from a table, whereas explicitly specifying column names fetches only the designated data fields.
- **When to prefer SELECT *:** It is best utilized during initial data exploration, ad-hoc analysis, or prototyping when the full structure of the table needs to be visualized.
- **When to prefer explicit columns:** Essential for production queries. It reduces system overhead, improves memory utilization, leverages indexing strategies, and ensures application resilience against changes in the baseline schema.

2. Column Aliasing and Table Structure

- **Keyword:** The `AS` keyword is used to assign an output name to a column.
- **Structural Impact:** No, an alias does not alter the physical table schema in the database. It is a temporary name evaluated purely at execution time to present the final query output.

3. Numeric Values in Quotes and Data Type Conflicts

- Enclosing a numeric value within single quotes (e.g., `'5000'`) evaluates it as a character string (VARCHAR or CHAR).
- Comparing a string literal to an integer or decimal column forces the query processor to perform an implicit data type conversion.
- **The Risk:** If the string contains unparsable characters, the query errors out at runtime. Additionally, implicit conversion often prevents the query optimizer from utilizing column indexes, resulting in full table scans and performance degradation.

4. ORDER BY Profit DESC vs. ASC

- **DESC (Descending):** Sorts values from highest to lowest. This sequence is mandatory when isolating top performers, as the highest positive numbers appear first.
- **ASC (Ascending):** Sorts values from lowest to highest. Utilizing this clause would mistakenly yield the least profitable or highest loss-making records.

5. LIMIT vs. T-SQL Equivalent and Syntax Variation

- **T-SQL Equivalent:** Microsoft SQL Server employs the `TOP` modifier or the ANSI-standard `OFFSET . . . FETCH` architecture.
- **Syntax Variation:** Relational systems evolved concurrently but independently before the widespread standardization of SQL. Engine vendors designed custom mechanisms to optimize internal sorting and row pagination, leading to proprietary syntax dialects.

6. Logical Execution Order

The query engine parses operations in a sequential layout distinct from written syntax:

```
[1] FROM → [2] WHERE → [3] SELECT → [4] ORDER BY → [5] LIMIT
```

Section B: Practical Task

1. Retrieve first 20 records

```
SELECT * FROM orders  
LIMIT 20;
```

2. Apply Column Aliasing

```
SELECT  
  `Order ID`,  
  `Order Date`,  
  Sales AS Total_Sales,  
  Profit  
FROM orders;
```

3. Filter High-Value Transactions

```
SELECT * FROM orders  
WHERE Sales > 5000;
```

4. Report Top 10 Most Profitable Orders

```
SELECT * FROM orders  
ORDER BY Profit DESC  
LIMIT 10;
```

Section C: Mini Project

Project Title:	Retail Profitability & Market Segment Analysis
Problem Statement:	Identify underperforming product categories and regions by analyzing the relationship between discount rates and net profit margins.
Dataset:	Sample Superstore Dataset (SampleSuperstore.csv)

```

CREATE DATABASE SuperstoreDB;
USE SuperstoreDB;

CREATE TABLE superstore_sales (
    Row_ID INT PRIMARY KEY,
    Order_ID VARCHAR(25),
    Order_Date DATE,
    Ship_Date DATE,
    Ship_Mode VARCHAR(25),
    Customer_ID VARCHAR(25),
    Customer_Name VARCHAR(255),
    Segment VARCHAR(50),
    Country VARCHAR(100),
    City VARCHAR(100),
    State VARCHAR(100),
    Postal_Code INT,
    Region VARCHAR(50),
    Product_ID VARCHAR(25),
    Category VARCHAR(100),
    Sub_Category VARCHAR(100),
    Product_Name VARCHAR(255),
    Sales DECIMAL(10, 2),
    Quantity INT,
    Discount DECIMAL(4, 2),
    Profit DECIMAL(10, 2)
);

SELECT
    Order_ID,
    Category,
    Sub_Category,
    Discount,
    Profit
FROM superstore_sales
WHERE Discount > 0.20
    AND Profit > 0
ORDER BY Discount DESC;

SELECT
    Region,
    Category,
    ROUND(SUM(Sales), 2) AS Total_Sales,
    ROUND(SUM(Profit), 2) AS Net_Profit,
    ROUND((SUM(Profit) / SUM(Sales)) * 100, 2) AS Profit_Margin_Percentage,
    ROUND(AVG(Discount) * 100, 2) AS Avg_Discount_Percentage
FROM superstore_sales
GROUP BY Region, Category
ORDER BY Region, Profit_Margin_Percentage ASC;

SELECT
    Region,
    Category,
    Sub_Category,
    COUNT(Row_ID) AS Total_Loss_Making_Orders,
    ROUND(SUM(Profit), 2) AS Total_Financial_Loss,
    ROUND(AVG(Discount) * 100, 2) AS Avg_Discount_Given
FROM superstore_sales

```

```
WHERE Profit < 0  
GROUP BY Region, Category, Sub_Category  
ORDER BY Total_Financial_Loss ASC;
```